

occlubio - Occlusion-Robust Face Recognition and Video Identification Platform

Project Report

Author: Shourya Pandey Institution: IIT (BHU) Date: 17 June 2026

1. Executive Summary

occlubio is a lightweight, low-latency face recognition platform with two phases:

- Enrollment (registration):** an end user creates an account and registers their face from uploaded photos or live webcam captures. Face quality is checked, an embedding is generated, duplicate faces are rejected, and the template is stored in a searchable database.
- Video identification (inference):** a video is uploaded; the system detects and tracks every face, matches each person against the enrolled database, and returns a structured report (name, ID, confidence, appearance timestamps, on-screen duration, number of appearances, unknown-person alerts) plus an annotated output video.

The recognition engine, a video-analytics layer, and a web platform (account + upload) were all built and verified end to end. The core research direction - recognizing faces through occlusion (masks, sunglasses, caps, profile) - is supported by a training pipeline that is ready to run on GPU hardware and is documented as the project's contribution.

This document is a standalone summary. Full design detail lives in `OCCLUSION_ROBUST_FR_ARCHITECTURE.md` (model/research deep-dive) and `PLATFORM.md` (platform/database/deployment).

2. Objectives

- Build a production-oriented, open-source face recognition system that is lightweight enough to deploy on NVIDIA Jetson edge devices and trainable on an NVIDIA DGX Spark.
- Support end-user self-registration and video-upload identification.
- Be robust to real-world conditions: partial occlusion, low light, motion blur, and pose.
- Keep the design modular so individual models (detector, recognizer) can be swapped or retrained.

3. System Architecture

The system is organized in three layers:

Layer	Responsibility	Key files
Recognition engine	detect, align, quality-gate, track, embed, search	occlubio/pipeline/, occlubio/gallery/, occlubio/tracking/
Analytics	per-track aggregation, timestamps, appearance/unknown reporting	occlubio/analytics/

Platform	accounts, enrollment, video jobs, web UI, database	occlubio/api/, occlubio/service/, occlubio/db/, web/
----------	--	--

Per-frame flow during identification:

```
video -> decode -> SCRFD face detection -> IoU tracking -> 5-point alignment
    -> quality gate (FIQA proxy) -> occlusion estimate -> face embedding (ArcFace)
    -> per-track embedding fusion -> 1:N FAISS search -> identity + confidence
    -> analytics (timestamps, appearances, unknown clustering) -> report + annotated video
```

Design principle: detection and tracking run on every processed frame (cheap), while heavy recognition runs once per track on quality-gated frames, with embeddings fused across the track for stability. The relational database is the source of truth; the FAISS vector index is a derived structure rebuilt from the database, so the vector store can be swapped without code changes.

4. Technology Stack and Model Selection

Stage	Component	Notes
Face detection	SCRFD (InsightFace)	bounding box + 5 landmarks in one pass; efficient, TensorRT/ONNX ready
Face alignment	5-point similarity transform (Umeyama)	canonical 112x112; identical in train and inference
Quality assessment	proxy FIQA (detector score + sharpness + exposure)	hook for CR-FIQA (CVPR 2023)
Face recognition	ArcFace (w600k_r50, InsightFace)	512-d embeddings; baseline. Occlusion-aware EdgeFace + AdaFace planned
Tracking	IoU tracker	hook for ByteTrack / DeepStream nvtracker
Similarity search	FAISS (cosine, IndexFlatIP)	scales to Milvus/Qdrant for millions of users
Web API	FastAPI + Uvicorn	auto OpenAPI docs, async background jobs
Database	SQLite (MVP) / PostgreSQL	users, enrollments, jobs

Full papers, repositories, benchmarks, and the rationale for each choice are in `OCCLUSION_ROBUST_FR_ARCHITECTURE.md`.

5. Enrollment (Registration) Module

Workflow:

1. The user registers an account (username, email, password). Passwords are hashed with PBKDF2.
2. The user submits one or more face images (file upload or live webcam capture).
3. For each image: detect the largest face, align it, and run the quality gate. Low-quality samples (blurry, dark, no face) are skipped.
4. Accepted samples are averaged into one robust 512-d template.

5. Duplicate-face detection: the template is searched against the existing database; if it matches a different user above a similarity threshold, the enrollment is rejected.
6. The template is stored in the database and the FAISS index is rebuilt.

6. Video Identification Module

A video is uploaded and processed as an asynchronous job. For each processed frame the engine detects, tracks, gates, embeds, and matches faces. The analytics layer then consolidates results:

- **Known people** are grouped by identity; appearances separated by short gaps are merged so the appearance count reflects real on-screen segments.
- **Unknown people** are filtered (short blips removed) and clustered by face-embedding similarity, so the same un-enrolled person is reported once rather than as many fragmented tracks.

Outputs: - JSON report: per person - ID, name, match confidence, first/last-seen timestamp, total on-screen duration, number of appearances; plus unknown-person alerts. - Annotated MP4: bounding boxes, names, tracking IDs, confidence scores, and a running timestamp. - Processing time: seconds taken, frames processed, processing FPS, and source video duration.

7. Database and Storage Design

Table	Key fields	Purpose
users	id, username (unique), email, password_hash	account
enrollments	user_id (unique), embedding (bytes), quality, n_images	one template per user
jobs	id, status, input_path, output_video, report_path	async video jobs

The FAISS index is labelled by username and rebuilt from the enrollments table on every change. Embeddings are stored as float32 bytes in the MVP; for production they should be encrypted at rest or replaced by a protected/cancelable template, because biometric data cannot be reissued if leaked.

8. Implemented Features (verified end to end)

Feature	Status	Evidence
Account registration + login (hashed passwords)	Done	HTTP test passed
Quality-gated enrollment	Done	low-quality samples skipped; quality score reported
Duplicate-face rejection	Done	second user enrolling the same face rejected (409)
Video identification job + report	Done	known people with confidence + timestamps
Unknown-person detection + clustering	Done	fragmented tracks collapsed into distinct people
Annotated output video (downloadable)	Done	valid MP4 served as attachment

Processing-time reporting	Done	seconds, frames, FPS in report
Web UI (register, enroll, webcam, upload, results)	Done	served at the site root

Example verified report (6-face sample, two users enrolled): 2 known people recognized with confidence 0.98 and 0.97, and 4 distinct unknown people detected - a clean result with no fragmentation.

9. Performance and Capability Assessment

Recognition quality depends on face size in pixels, not absolute distance.

Face size (longer side)	Behavior
≥ 150 px	excellent
100-150 px	good
60-100 px	marginal
< 60 px	unreliable

For a typical 1080p camera (with frames internally capped to 1280 px wide), the practical sweet spot is roughly 0.5 to 2.5 metres.

The current baseline recognizer (ArcFace) works well on frontal, well-lit faces and tolerates transparent eyeglasses and mild pose. It is not yet trained for heavy occlusion: masks, sunglasses, strong profile, low light, motion blur, and very small faces all reduce accuracy. The heuristic occlusion estimator can roughly tag mask/sunglasses/cap regions, but detecting occlusion is distinct from recognizing through it.

Latency: on CPU the system runs at well under 1 FPS (suitable for offline analysis with frame striding). On a Jetson Orin NX with TensorRT FP16/INT8 the per-face path is expected to be about 8 to 15 ms, enabling real-time multi-camera operation. FAISS 1:N search is sub-millisecond for up to one million identities.

10. Limitations and Research Contribution

The baseline does not yet recognize through significant occlusion. The intended contribution is an **occlusion-aware recognizer**: a lightweight EdgeFace backbone trained with the AdaFace quality-adaptive loss and heavy synthetic-occlusion augmentation (masks, sunglasses, caps, scarves, low light, motion blur), optionally distilled from a strong teacher. The augmentation utilities and the training script are already implemented; the exported model plugs into the inference pipeline through a configuration setting. Training requires a face dataset and GPU hardware (DGX Spark).

11. How to Run

```
# 1. Environment (Python 3.10-3.12; 3.14 has no wheels for the CV stack)
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -e ".[infer,api]"

# 2. Web platform
uvicorn occlubio.api.app:app --host 0.0.0.0 --port 8000
#   UI:          http://localhost:8000
#   API docs:    http://localhost:8000/docs
```

```
# 3. Command-line (without the web UI)
python scripts/enroll.py --images data/enroll --gallery gallery_store
python scripts/identify_video.py --source data/my_video.mp4 --gallery gallery_store \
    --out data/out.mp4 --report data/out.json --stride 2 --no-display
```

Annotated videos use the mp4v codec; open them in a media player such as VLC (browsers cannot preview mp4v inline).

12. Implementation Roadmap

1. (Done) Working engine, analytics, and web platform: register, enroll, identify, report.
2. Token authentication and per-user job history; containerization; PostgreSQL.
3. Background worker queue (Celery + Redis) and object storage for scalable video jobs.
4. Train and integrate the occlusion-aware recognizer; add anti-spoofing.
5. Vector database (Milvus) at scale; encrypted/protected templates; GPU serving (Triton) or Jetson + DeepStream edge deployment.
6. Monitoring, bias evaluation, and legal review before any real deployment.

13. Responsible Use, Security, and Privacy

This is a biometric system and is legally sensitive (EU AI Act; India DPDP Act 2023). Before any real deployment: obtain consent and a data-protection impact assessment, encrypt or protect stored templates, add face anti-spoofing, enable HTTPS and real authentication, support data deletion, and evaluate accuracy across demographic groups. Generative face restoration, if added, must never be used to fabricate identity evidence.

14. References

- InsightFace (SCRFD, ArcFace): <https://github.com/deepinsight/insightface>
- AdaFace (CVPR 2022): <https://github.com/mk-minchul/AdaFace>
- EdgeFace (IEEE T-BIOM 2024): <https://github.com/otroshi/edgeface>
- CR-FIQA (CVPR 2023): <https://github.com/fdbtrs/CR-FIQA>
- ByteTrack (ECCV 2022): <https://github.com/FoundationVision/ByteTrack>
- FAISS: <https://github.com/facebookresearch/faiss>
- FastAPI: <https://github.com/tiangolo/fastapi>
- NVIDIA DeepStream / TensorRT / Triton (deployment target)

A fuller curated index of papers and repositories is in `OCCLUSION_ROBUST_FR_ARCHITECTURE.md`.

15. Appendix: Repository Structure

occlubio/	recognition engine, analytics, training, db, service, api
pipeline/	detector, aligner, quality, occlusion, antispoof, recognizer, engine
gallery/	FAISS 1:N store
tracking/	IoU tracker
analytics/	identity log + report
training/	AdaFace head, dataset, train + ONNX export
db/ service/ api/	platform layer
scripts/	enroll, identify_video, recognize_video, benchmark, make_demo, build_report
web/	single-page UI
deepstream/	edge deployment config templates

```
configs/          default.yaml
tests/           smoke tests
OCCLUSION_ROBUST_FR_ARCHITECTURE.md  model/research deep-dive
PLATFORM.md      platform/database/deployment design
REPORT.md        this report
```